How to Create a Docker File

Reference : RA 448

Description

Docker is a powerful platform that enables developers to build, package, and distribute applications in standardized environments called containers. This guide walks you through the process of creating and managing Docker containers effectively.

What is a Docker?

Docker is a tool designed to create, deploy, and run applications within self-contained units called **containers**. Containers allow applications to be packaged with all their dependencies, making them portable and scalable across environments.

- A container is dynamically created from a Docker **image** that is a template (available as an archive file) for this container.
- Creating a Docker image requires a **Dockerfile**.
- This Dockerfile is a text file describing the **contents** of the image.

> **TIP**
>
> See the Docker Workflow Diagram for a visual overview of the Docker build and deployment process.

Prerequisites

Installing Docker on WSL

If you're using **WSL (Windows Subsystem for Linux)**, follow our detailed guide:
Installing Docker on WSL - Complete step-by-step installation guide

For general WSL installation, refer to Microsoft's documentation:
Install WSL

If you are installing WSL into a VM, you have to enabled the Virtualization. For more details please refer to this page Nested VM

To run Docker on WSL2 without Docker Desktop, consider this guide:
Running Docker on WSL2 - DEV

In this Reference Architecture

The following articles explain:

- Details about the Dockerfile describing the contents of a Docker image.
- How to create a Docker image and save it as an archive file.

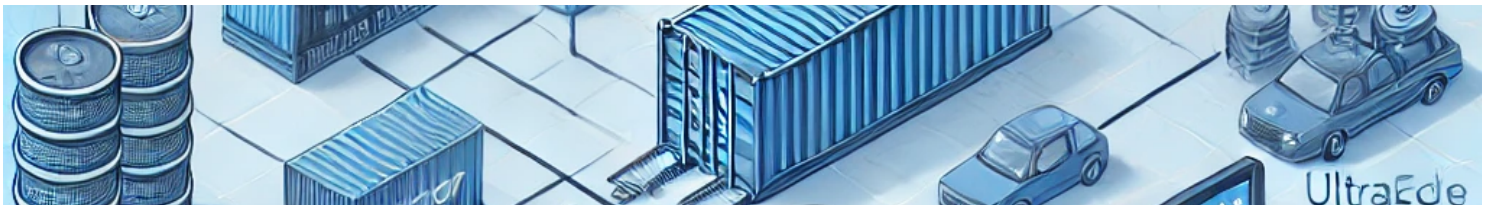
Going Further

- Follow this link to manipulate a Docker image for UltraEdge2000 with C#.
- Follow this link to work with Docker images for UltraEdge2000 in IGXL/VBT.
- Finally, this article explains how to encrypt a Docker archive to send to the UltraEdge2000.

Conclusion

Docker simplifies the deployment and management of applications in a portable and scalable way. By following this guide, you can create, manage, and share Docker containers with ease.

TIP For more info: Official Docker Documentation

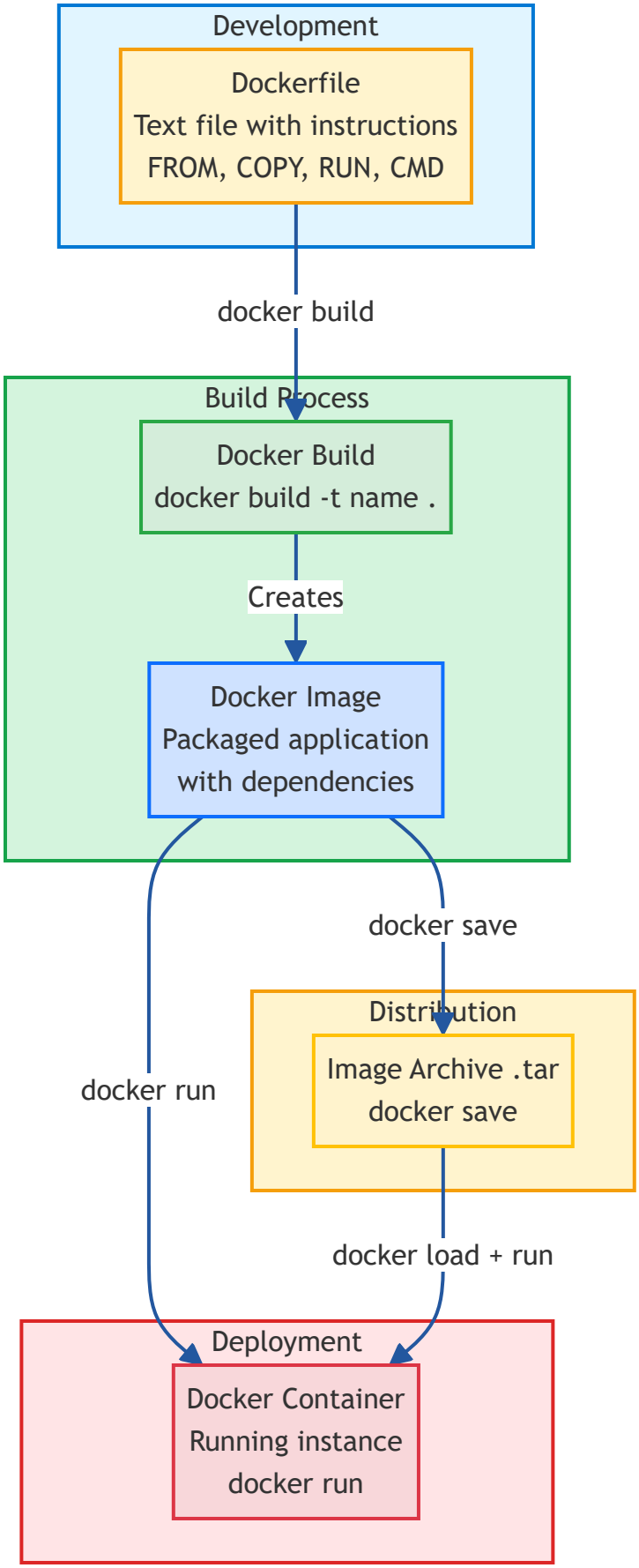


Docker Workflow - Visual Overview

This page provides a visual representation of the Docker build and deployment workflow, from creating a Dockerfile to running a container.

Docker Workflow Diagram

> **TIP**
>
> Click on any stage to jump to its detailed explanation below.



Workflow Stages Explained

Development

Dockerfile - A text file containing instructions that define:

- Base image (`FROM`)
- Files to copy (`COPY`)
- Commands to execute (`RUN`)
- Startup command (`CMD`)

↑ Back to diagram

Build Process

Docker Build - Executes the Dockerfile instructions:

```
docker build -t myapp:latest .
```

Creates a **Docker Image** - A packaged application with all dependencies included.

↑ Back to diagram

Distribution

Image Archive - Export the image for sharing or deployment:

```
docker save myapp:latest > myapp.tar
```

↑ Back to diagram

Deployment

Docker Container - A running instance of the image:

```
docker run -d myapp:latest
```

Or load from archive:

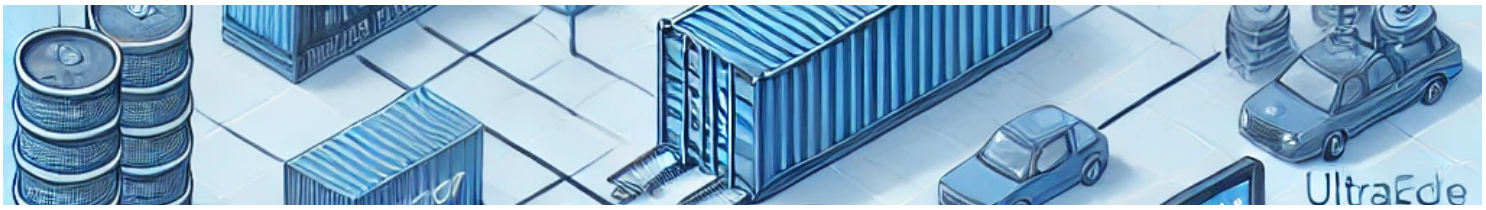
```
docker load < myapp.tar
docker run -d myapp:latest
```

↑ Back to diagram

Next Steps

- Installing Docker on WSL - Visual installation guide
- Advanced Dockerfile Explanation - Detailed Dockerfile concepts
- Creating Docker Images - Step-by-step image creation

This documentation is part of the Teradyne Archimedes Reference Architecture series.



Installing Docker on WSL (Windows Subsystem for Linux)

This guide provides step-by-step instructions for installing Docker on WSL without using Docker Desktop.

> **TIP**
>
> Before starting, review the Installation Overview to see the complete installation flow.

Prerequisites

1. Install WSL 2

First, ensure WSL 2 is installed and configured on your Windows system.

```
# Install WSL with Ubuntu (latest LTS version - recommended)
wsl --install

# Or install a specific Ubuntu version if needed
wsl --install -d Ubuntu-22.04
wsl --install -d Ubuntu-24.04
```

> **NOTE**
>
> Using `wsl --install` without specifying a distribution installs Ubuntu (latest LTS) by default, which is recommended for most users.

> **NOTE**
>
> You may need to restart your computer after installing WSL.

2. Verify WSL Version

Ensure you're using WSL 2 (not WSL 1):

```
wsl --list --verbose
```

If your distribution is running WSL 1, upgrade it:

```
# Replace 'Ubuntu' with your distribution name if different
wsl --set-version Ubuntu 2
```

3. Set WSL 2 as Default

```
wsl --set-default-version 2
```

Installing Docker on WSL

Once WSL 2 is ready, open your WSL terminal (e.g., Ubuntu) and follow these steps:

Step 1: Update Package Index

```
sudo apt update
```

```
sudo apt upgrade -y
```

Step 2: Install Required Dependencies

```
sudo apt install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

Step 3: Add Docker's Official GPG Key

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -
o /etc/apt/keyrings/docker.gpg
```

Step 4: Set Up the Docker Repository

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Step 5: Install Docker Engine

```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

> **IMPORTANT**
> Wait for the installation to complete. You should see "Processing triggers" and no error messages. If you see errors about packages not being available, verify Step 4 was completed correctly.

Verify the installation:

Check if dockerd is installed:

```
which dockerd
```

Should output: `/usr/bin/dockerd`

Check Docker version:

```
docker --version
```

Should output something like: `Docker version 24.x.x, build xxxxx`

Step 6: Start Docker Service

WSL doesn't use systemd by default, so we need to start Docker manually.

Option A: Start Docker manually (works on all WSL versions)

```
sudo dockerd &
```

> **NOTE**
> The `&` runs Docker in the background. You'll see some startup logs, press Enter to get your prompt back.

Option B: Enable systemd (Ubuntu 22.04+ only)

If you're using Ubuntu 22.04 or later, you can enable systemd:

1. Edit the WSL configuration:

```
sudo nano /etc/wsl.conf
```

2. Add the following content:

```
[boot]
systemd=true
```

3. Save the file (Ctrl+X, Y, Enter) and exit WSL:

```
exit
```

4. Shutdown WSL from Windows PowerShell:

```
wsl --shutdown
```

5. Restart WSL:

```
wsl
```

6. Start Docker:

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Step 7: Add User to Docker Group

First, verify the docker group exists (it should be created automatically):

```
getent group docker
```

If the group doesn't exist, create it:

```
sudo groupadd docker
```

Add your user to the docker group:

```
sudo usermod -aG docker $USER
```

Log out and back in for the changes to take effect:

```
exit
```

Then reopen WSL and restart Docker (use Option A or B from Step 6).

Step 8: Verify Docker Installation

Wait a few seconds for Docker to start, then verify it's working:

```
docker run hello-world
```

If successful, you should see "Hello from Docker!" message.

> **TIP**
> If using Option A (manual start), you need to run `sudo dockerd &` each time you start WSL. See the Configuration Guide for auto-start options.

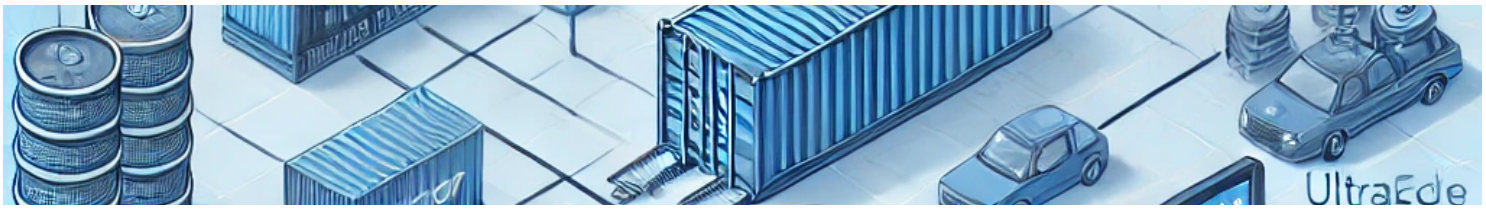
Next Steps

- Configure Docker on WSL - Auto-start, passwordless sudo, and testing
- Troubleshooting Docker on WSL - Common issues and solutions
- Learn how to create Dockerfiles
- Explore Docker Compose for multi-container applications

Additional Resources

- Official Docker Installation Page
- Official Docker Documentation
- WSL Documentation
- Docker on WSL 2 Best Practices
- Overview of Docker Installation

This documentation is part of the Teradyne Archimedes Reference Architecture series.



Docker on WSL - Post-Installation Configuration

This guide covers optional configurations to improve your Docker on WSL experience.

Enable Docker Without sudo

To run Docker commands without `sudo`, add your user to the `docker` group:

```
sudo usermod -aG docker $USER
```

> **IMPORTANT**
>
> You must **log out and log back in** (or restart WSL) for the group changes to take effect.

Verify the change by closing and reopening WSL terminal, then run:

```
docker run hello-world
```

Auto-Start Docker on WSL Startup

Choose one of the following methods to automatically start Docker when WSL launches.

Option 1: Auto-start with .bashrc (All WSL versions)

Add this to your `~/.bashrc`:

```
# Start Docker daemon if not running
if ! pgrep -x dockerd > /dev/null; then
    echo "Starting Docker..."
    sudo dockerd > /dev/null 2>&1 &
    sleep 2
fi
```

Apply the changes:

```
echo '
# Start Docker daemon if not running
if ! pgrep -x dockerd > /dev/null; then
    echo "Starting Docker..."
    sudo dockerd > /dev/null 2>&1 &
    sleep 2
fi' >> ~/.bashrc


source ~/.bashrc
```

> **NOTE**
> You may need to configure passwordless sudo for dockerd. See below.

Option 2: Using systemd (Ubuntu 22.04+ only)

If you enabled systemd in Step 6, Docker will start automatically. No additional configuration needed!

If your WSL distribution supports systemd:

1. Edit `/etc/wsl.conf`:

```
sudo nano /etc/wsl.conf
```

2. Add the following:

```
[boot]
systemd=true
```

3. Restart WSL:

```
wsl --shutdown


wsl
```

4. Enable Docker to start automatically:

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

Configure Passwordless sudo for Docker

To avoid entering your password when starting Docker:

```
echo "$USER ALL=(ALL) NOPASSWD: /usr/bin/dockerd" | sudo tee /etc/sudoers.d/dockerd
```

> **WARNING**
>
> This allows running `dockerd` without a password. Only do this on trusted systems.

Testing Your Docker Installation

Run a Simple Container

```
docker run -d -p 8080:80 nginx
```

Access it from your Windows browser at: `http://localhost:8080`

Build a Test Image

Create a simple Dockerfile:

```
mkdir ~/docker-test
```

```
cd ~/docker-test
```

```
nano Dockerfile
```

Add the following content:

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y curl
CMD ["echo", "Docker on WSL is working!"]
```

Build and run:

```
docker build -t test-image .
```

```
docker run test-image
```

Useful Docker Commands

```
# Check Docker version
docker --version

# List running containers
docker ps

# List all containers (including stopped)
docker ps -a

# List images
docker images

# Stop a container
docker stop <container_id>

# Remove a container
docker rm <container_id>

# Remove an image
docker rmi <image_id>

# View Docker logs
docker logs <container_id>

# Execute command in running container
docker exec -it <container_id> /bin/bash
```

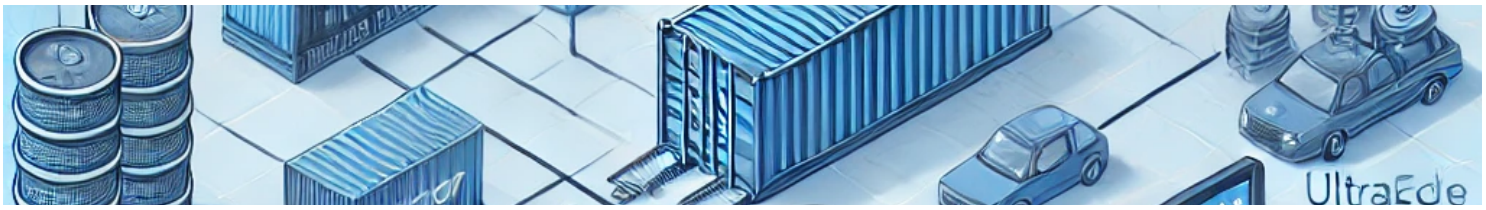
Next Steps

- Troubleshooting Docker on WSL
- Learn how to create Dockerfiles
- Explore Docker Compose for multi-container applications
- Read about Docker best practices

Additional Resources

- Official Docker Documentation
- WSL Documentation
- Docker on WSL 2 Best Practices

This documentation is part of the Teradyne Archimedes Reference Architecture series.



Docker on WSL - Troubleshooting Guide

This guide helps you resolve common issues with Docker on WSL.

Common Issues and Solutions

Issue 1: "dockerd: command not found" or "group 'docker' does not exist"

Cause: Docker was not installed successfully in Step 5.

Solution:

1. Verify the Docker repository was added correctly:

```
cat /etc/apt/sources.list.d/docker.list
```

You should see a line starting with `deb [arch=...] https://download.docker.com/linux/ubuntu`

2. If the file doesn't exist or is empty, redo Steps 3-4:

Step 3:

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -
o /etc/apt/keyrings/docker.gpg
```

Step 4:

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3. Then reinstall Docker:

```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

4. Verify installation:

```
which dockerd
```

```
docker --version
```

Issue 2: "Cannot connect to the Docker daemon"

Solution: Start the Docker service:

```
sudo dockerd &
```

Issue 3: "Permission denied while trying to connect to the Docker daemon socket"

Solution: Ensure your user is in the `docker` group:

```
sudo usermod -aG docker $USER
```

Log out and log back in for the changes to take effect.

Issue 4: Docker service won't start

Solution: Check Docker service status and logs:

```
sudo service docker status
```

```
sudo journalctl -u docker
```

Issue 5: WSL runs out of memory

Solution: Create or edit `.wslconfig` in your Windows user folder (`C:\Users\<YourName>\.wslconfig`):

```
[wsl2]
memory=4GB
processors=2
swap=2GB
```

Restart WSL:

```
wsl --shutdown
```

```
wsl
```

Clean Installation - Removing WSL and Starting Over

If you encounter issues and want to start fresh, you can completely remove WSL and reinstall.

Option 1: Remove Only the Ubuntu Distribution (Recommended)

This keeps WSL installed but removes your Ubuntu installation and all its data.

From PowerShell (as Administrator):

List all installed distributions:

```
wsl --list --verbose
```

Unregister Ubuntu (this deletes all data in that distribution):

```
wsl --unregister Ubuntu
```

Or for specific versions:

```
wsl --unregister Ubuntu-22.04
```

Or:

```
wsl --unregister Ubuntu-24.04
```

> **WARNING**
> This will permanently delete all files, configurations, and Docker installations in that WSL distribution. Make sure to backup any important data first.

After unregistering, you can install a fresh Ubuntu distribution:

```
wsl --install -d Ubuntu
```

Then follow the installation guide from the beginning.

Option 2: Completely Uninstall WSL

If you want to remove WSL entirely from Windows:

From PowerShell (as Administrator):

1. Unregister all distributions:

List all distributions:

```
wsl --list
```

Unregister each one:

```
wsl --unregister Ubuntu
```

```
wsl --unregister Ubuntu-22.04
```

Repeat for each distribution listed.

2. Uninstall WSL:

Disable WSL features:

```
dism.exe /online /disable-feature /featurename:Microsoft-Windows-Subsystem-Linux /norestart
```

```
dism.exe /online /disable-feature /featurename:VirtualMachinePlatform /norestart
```

3. Restart your computer

4. To reinstall WSL, follow the guide from the Prerequisites section.

Quick Reset Without Losing WSL

To clean up Docker while preserving your WSL setup:

From your WSL terminal:

Remove Docker and all its data:

```
sudo apt remove --purge docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

```
sudo rm -rf /var/lib/docker
```

```
sudo rm -rf /var/lib/containerd
```

Remove Docker group:

```
sudo groupdel docker
```

Remove repository configuration:

```
sudo rm /etc/apt/sources.list.d/docker.list
```

```
sudo rm /etc/apt/keyrings/docker.gpg
```

Update package list:

```
sudo apt update
```

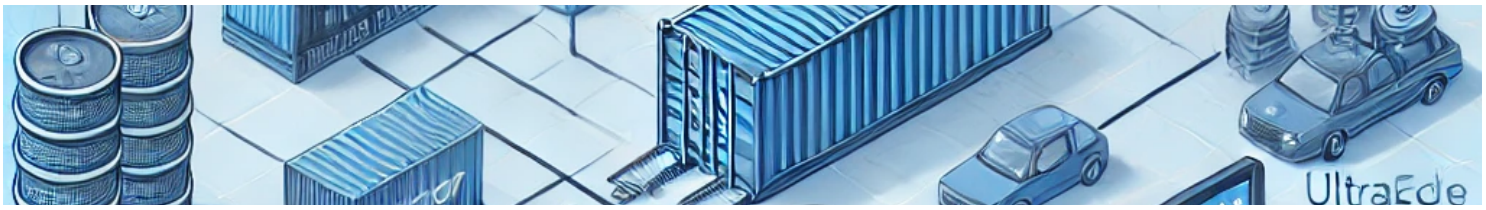
Then start fresh from Step 3 of the installation guide.

Getting Help

If you continue to experience issues:

1. Check the Official Docker Documentation
2. Review WSL Documentation
3. Search Docker on WSL 2 Best Practices
4. Consult with your team's DevOps engineers

This documentation is part of the Teradyne Archimedes Reference Architecture series.

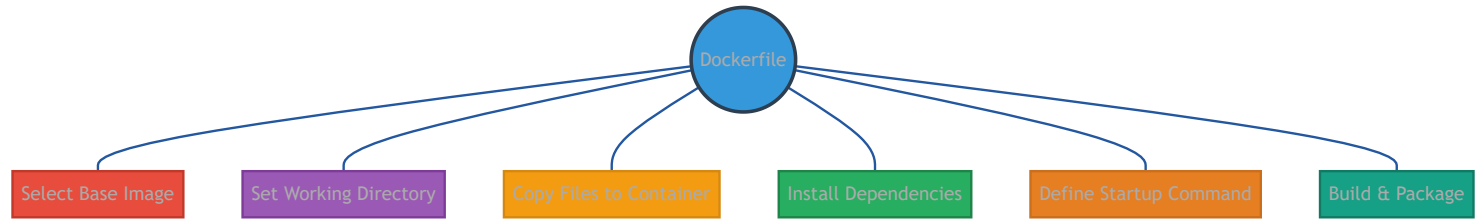


Advanced Docker File Explanation

This page provides a detailed explanation of the `Dockerfile` used in the RA_FAARCH_448_CreateDocker reference. We'll break down each instruction and explain its purpose. Additionally, you can directly download the full `Dockerfile` and the `makedocker.sh` script used to build your Docker image.

Dockerfile Concepts Mindmap

> **TIP**
> Click on any node to jump directly to its detailed explanation below.



Dockerfile Breakdown

The `Dockerfile` is a set of instructions that Docker uses to build an image. Here's a common structure and what each section typically does:

Base Image

```
FROM python:3.8-slim
```

- This sets the **base image** for your container. `python:3.8-slim` provides a lightweight Python environment.

↑ Back to diagram

Working Directory

```
WORKDIR /app
```

- Defines the working directory inside the container. Any subsequent commands (like `COPY` or `RUN`) operate relative to this directory.

↑ Back to diagram

Copy Files

```
COPY script.py ./
```

- Copies `script.py` from your host into the container's `/app` directory. ↑ Back to diagram

Install Dependencies (optional section)

```
RUN pip install --no-cache-dir -r requirements.txt
```

- Installs Python dependencies inside the container if a `requirements.txt` file exists.

↑ Back to diagram

Run Command

```
CMD ["python", "script.py"]
```

- This defines the default command that will run when the container starts. Here, it launches the Python script.

↑ Back to diagram

Docker Build Script

The `makedocker.sh` script automates Docker image creation, compression, cleanup, and verification.

Download the Files

- Download Dockerfile
- Download makedocker.sh

What makedocker.sh Does

1. Builds a Docker image
2. Compresses it into `.tar.gz`
3. Cleans up the image to save space
4. Lists remaining Docker images for verification

↑ Back to diagram

Running the Script

To execute the script:

```
chmod +x makedocker.sh
./makedocker.sh
```

This generates `sampleapp.tar.gz`, a ready-to-deploy Docker image.

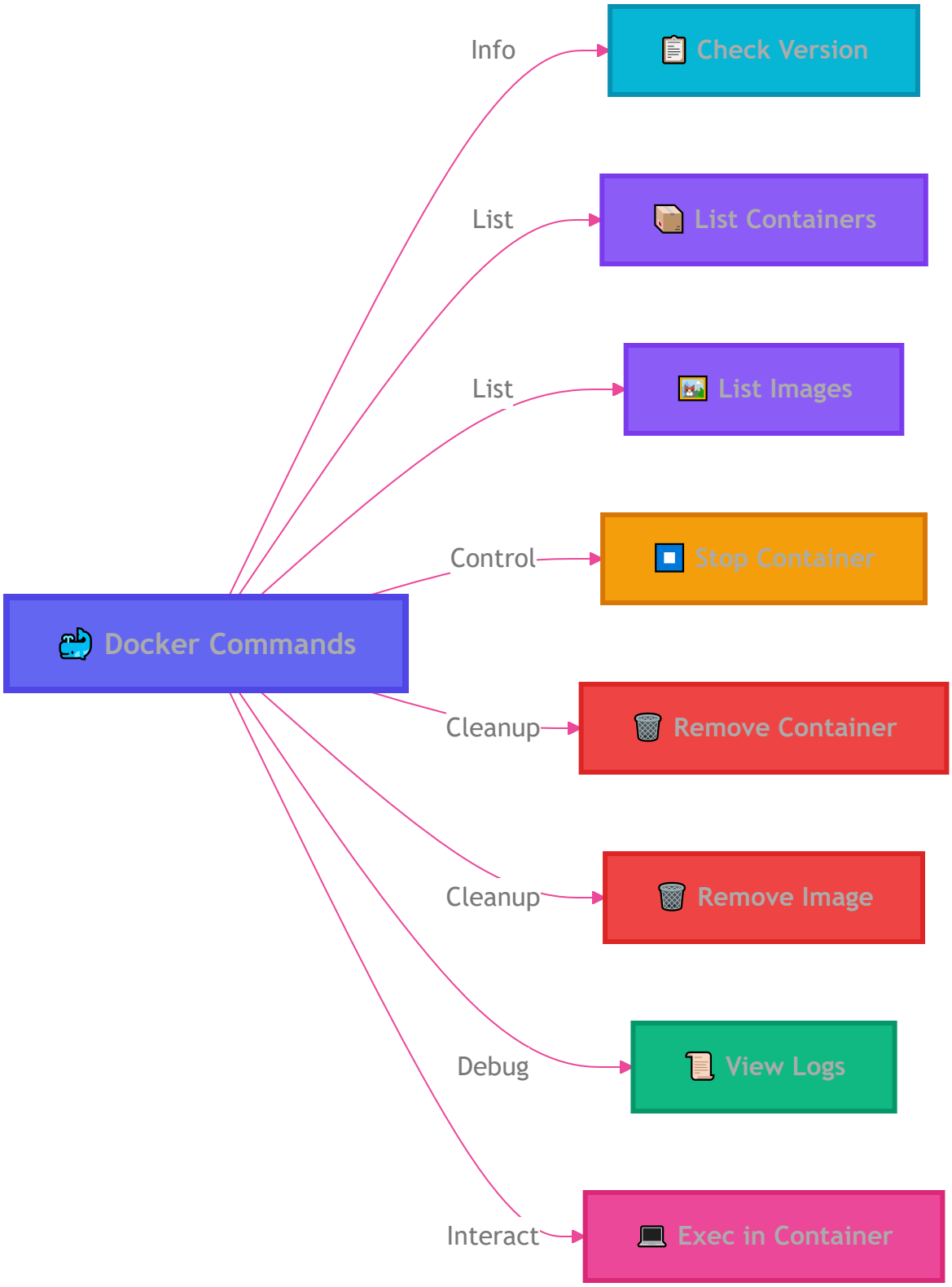
Conclusion

This detailed view should help you understand how each part of the Dockerfile contributes to building and running your container. The shell script complements it by making the build process simple and repeatable.

For additional customization, modify the `Dockerfile` to suit your application requirements.

Useful Docker Commands

Mind Map



1. Check Docker Version

Example:

```
docker --version
```

```
check_docker_version() {
    docker --version
}
```

⬆️ Back to Mind Map

2. List Running Containers

Example:

```
docker ps
```

```
list_running_containers() {
    docker ps
}
```

⬆️ Back to Mind Map

3. List All Containers

Example:

```
docker ps -a
```

```
list_all_containers() {
    docker ps -a
}
```

⬆️ Back to Mind Map

4. List Images

Example:

```
docker images
```

```
list_docker_images() {
    docker images
}
```

⬆️ Back to Mind Map

5. Stop a Container

Example:

```
docker stop <container_id>
```

```
stop_container() {
    docker stop "$1"
}
# Usage: stop_container <container_id>
```

⬆️ Back to Mind Map

6. Remove a Container

Example:

```
docker rm <container_id>
```

```
remove_container() {
    docker rm "$1"
}
# Usage: remove_container <container_id>
```

⬆️ Back to Mind Map

7. Remove an Image

Example:

```
docker rmi <image_id>
```

```
remove_image() {
    docker rmi "$1"
```

```
}
# Usage: remove_image <image_id>
```

⬆️ Back to Mind Map

8. View Docker Logs

Example:

```
docker logs <container_id>


view_container_logs() {
    docker logs "$1"
}
# Usage: view_container_logs <container_id>
```

⬆️ Back to Mind Map

9. Exec in Container

Example:

```
docker exec -it <container_id> /bin/bash


exec_in_container() {
    docker exec -it "$1" /bin/bash
}
# Usage: exec_in_container <container_id>
```

⬆️ Back to Mind Map

Source Code Downloads

Below is a list of available source files for download, along with a brief description of each file.

File Name	Download Link
Entire Solution	The solution
GIT Hub Link	Git Hub

More details about those source code here